

RAPPORT DE PROJET

Système de Gestion E-Commerce Full-Stack

Auteur	Anas
Formateur	Khalid MZIBRA
Module	DOWFS201 — Projet de synthèse
Date	Mai 2026
Technologies	Laravel 12 · React 19 · MySQL/SQLite

1. Introduction

Ce rapport présente la conception et la réalisation d'une plateforme e-commerce complète, développée dans le cadre du module **DOWFS201 — Projet de synthèse**. L'objectif est de créer une application web moderne, sécurisée et multilingue permettant la gestion des ventes en ligne.

Objectifs principaux

- Concevoir une API REST robuste avec Laravel 12 couvrant le cycle e-commerce complet.
- Développer une SPA réactive avec React 19, Vite et Tailwind CSS.
- Implémenter un système de rôles : Administrateur, Vendeur, Client, Caissière de Charge.
- Gérer le panier pour les utilisateurs invités et authentifiés (fusion automatique).
- Intégrer le paiement en ligne (PayPal) et la génération de factures PDF (DomPDF).
- Proposer une interface multilingue : Anglais, Français et Arabe (support RTL).
- Mettre en place un chatbot d'assistance et un système de recommandations ML.

2. Analyse des Besoins

L'analyse des besoins identifie les fonctionnalités attendues pour chaque acteur du système. Quatre rôles distincts sont définis, chacun avec des permissions spécifiques.

Rôles et Acteurs

Rôle	Description	Accès principal
Client	Utilisateur final qui parcourt et achète des produits.	Catalogue, panier, commandes, avis, liste de souhaits
Caissière de Charge	Opératrice chargée du traitement des paiements et de la validation des commandes.	Paiements, validation commandes, émission factures, retours
Vendeur	Marchand qui publie et gère son catalogue de produits.	CRUD produits, images, stock, commandes reçues
Administrateur	Gestionnaire global de la plateforme.	Utilisateurs, catégories, analytics, modération, tous produits

Besoins Fonctionnels

N°	Fonctionnalité	Priorité
F1	Authentification sécurisée (Sanctum) et gestion de sessions	Haute
F2	Catalogue produits avec filtres, pagination et recherche	Haute
F3	Panier invité/authentifié avec fusion à la connexion	Haute
F4	Processus de commande avec adresse, livraison et coupon	Haute
F5	Paiement sécurisé (PayPal) + paiement à la livraison	Haute
F6	Génération automatique de factures PDF (DomPDF)	Haute
F7	Tableau de bord Admin avec analytics et graphiques	Moyenne
F8	Interface multilingue EN / FR / AR avec RTL	Moyenne
F9	Chatbot d'assistance + recommandations ML	Optionnel

3. Conception

Architecture Générale

L'application adopte une architecture **client-serveur découplée (Headless)** avec une séparation nette entre le backend API REST (Laravel) et le frontend SPA (React). Cette approche favorise la scalabilité et la maintenabilité.

Couche	Composant	Rôle
Présentation	React 19 + Vite + Redux Toolkit	SPA avec état centralisé
Communication	Axios + API REST /api/v1	Échanges JSON + Bearer token
Logique Métier	Laravel 12 (Controllers, Middleware)	Validation, autorisation, traitement
Persistance	Eloquent ORM + SQLite/MySQL	14 modèles, 16 migrations
Stockage Fichiers	Laravel Storage (disk:public)	Images, avatars, PDFs factures
Auth	Laravel Sanctum	Tokens Bearer + gestion invités

Modèle de Données

La base de données comprend **14 modèles Eloquent** liés par des relations HasMany, BelongsTo et BelongsToMany.

Modèle	Champs clés	Relations
User	id, name, email, role, is_active	orders, cart, addresses, reviews
Product	id, name, slug, price, stock, category_id	images, reviews, orderItems
Category	id, name, slug, parent_id (auto-réf.)	products, children
Order	id, order_number, status, total, user_id	items, address, shippingMethod
OrderItem	id, order_id, product_id, quantity, price	order, product
Cart/CartItem	id, user_id?, session_id? / cart_id, product_id	user, items, product
Address	id, user_id, city, country, is_default	user, orders
Review	id, user_id, product_id, rating, is_approved	user, product
Coupon	id, code, type, value, expires_at	—
ShippingMethod	id, name, price, estimated_days_min/max	orders

4. Réalisation

4.1 Stack Technique

Technologie	Version	Usage
Laravel	12	API REST, ORM, validation, middleware
React	19	Interface SPA, composants fonctionnels
Vite	8	Build ultra-rapide + HMR
Redux Toolkit	—	Slices : auth, cart, theme, ui
Tailwind CSS	4	Design system utilitaire
Laravel Sanctum	—	Authentification par token Bearer
DomPDF	—	Génération factures PDF
i18next	—	Internationalisation EN/FR/AR
PayPal SDK	—	Paieement en ligne sécurisé
SQLite / MySQL	—	Développement local / production

4.2 API REST — Endpoints Principaux

Méthode	Endpoint	Rôle	Accès
POST	/api/v1/login	Connexion + token	Public
GET	/api/v1/products	Catalogue + filtres	Public
POST	/api/v1/cart/add	Ajouter au panier	Public
POST	/api/v1/orders	Créer une commande	Auth
GET	/api/v1/orders/{id}/invoice	Télécharger facture PDF	Auth
GET	/api/v1/seller/products	Produits du vendeur	Vendeur
GET	/api/v1/admin/dashboard	Statistiques globales	Admin
PUT	/api/v1/admin/orders/{id}/status	Maj statut commande	Admin/Caissière
GET	/api/v1/admin/analytics/sales	Analytics ventes	Admin

4.3 Pages Frontend

Route	Page	Description
/	HomePage	Bannière, produits vedettes, recommandations
/products	ProductsPage	Catalogue avec filtres et pagination

Route	Page	Description
/cart	CartPage	Panier + coupon + résumé
/checkout	CheckoutPage	Adresse, livraison, paiement (protégé)
/orders	OrdersPage	Historique + téléchargement facture (protégé)
/admin/*	Admin (5 pages)	Dashboard, produits, commandes, utilisateurs, catégories
/seller/*	Seller (4 pages)	Dashboard, produits, formulaire, commandes

5. Tests et Installation

5.1 Tests

Le projet utilise **PHPUnit** (intégré à Laravel) pour les tests unitaires et fonctionnels. Les commandes ci-dessous permettent de lancer la suite complète.

```
php artisan test
```

5.2 Installation locale

Backend

```
cd backend && composer install
cp .env.example .env && php artisan key:generate
php artisan migrate --seed
php artisan storage:link
php artisan serve → http://localhost:8000
```

Frontend

```
cd frontend && npm install
npm run dev → http://localhost:5173
```

5.3 Comptes de test

Rôle	Email	Mot de passe
Administrateur	admin@example.com	password
Vendeur	seller@example.com	password
Client	client@example.com	password
Caissière de Charge	caissiere@example.com	password

6. Conclusion

Ce projet de plateforme e-commerce constitue une application web complète et fonctionnelle, répondant aux standards modernes du développement full-stack. Il démontre la maîtrise d'une architecture découplée (API + SPA) avec un système de rôles granulaire incluant le rôle de **Caissière de Charge**.

Points forts réalisés

- Architecture découplée REST + SPA facilitant la maintenance et l'évolution.
- Système de rôles complet : Admin / Vendeur / Client / Caissière de Charge.
- Support multilingue robuste (EN / FR / AR) avec direction RTL automatique.
- Panier hybride (invité + authentifié) avec fusion automatique à la connexion.
- Intégration PayPal et génération de factures PDF (DomPDF).
- Chatbot d'assistance avec détection d'intention en plusieurs langues.
- Tableau de bord analytique avec graphiques Recharts en temps réel.

Pistes d'amélioration

- Migration vers MySQL en production avec optimisation des index.
- Notifications temps réel via WebSockets (Laravel Echo + Pusher).
- Application mobile React Native partageant la même API.
- Déploiement CI/CD sur Railway ou Vercel.

Rapport généré automatiquement — DOWFS201 · Projet E-Commerce · Mai 2026